◈ ScenePilot AI — Project Specification

IBM SkillsBuild AI Builders Challenge · July 2025 · Built with IBM Bob · Version 2.0 (Hallucination Verifier Added)

PROJECT TYPE	PRIMARY STACK	SAFETY LAYER	LLM PROVIDERS
Multi-Agent AI SaaS	LangGraph · FastAPI · React	IBM Granite Guardian 3-8b	Groq · Gemini · watsonx.ai

PIPELINE AGENTS	STATUS
7 agents	Production Ready

1. Project Overview

ScenePilot AI is an AI-powered branching narrative generator with a multi-agent quality-gate pipeline. It takes a story premise, genre, and tone and produces a fully validated, content-safe branching narrative graph — structurally correct, style-compliant, hallucination-checked, and ready for export to JSON, Markdown, Twine, or a 3D game engine blueprint.

Core Capabilities

- 7-agent LangGraph pipeline with automatic self-correction retry loop
- RAG-based hallucination verification (confidence + grounding + evidence)
- IBM Granite Guardian content safety scanning via watsonx.ai
- Four-mode diff-patch repair engine (82% token reduction vs full regeneration)
- Three-layer token budget gate (pre-flight · router · generator)
- Real-time SSE progress stream per pipeline run
- Interactive story tree, game engine blueprint, playable emulator, 3-format export
- Prometheus observability + Grafana dashboard

2. Problem Statement

Enterprise Flaws in Standard LLM Pipelines

Flaw	Description	Business Impact

Unpredictable API costs	No pre-flight budget estimation. LLM calls proceed regardless of token budget	Unbounded API bills; SaaS margin destruction
Infinite repair loops	Failed validation triggers full story regeneration every retry (~9,000 tokens)	Exponential cost escalation; frequent 429 quota exhaustion
Stateless regeneration	Each retry discards the previous story entirely	100% token waste on minor structural issues
Brittle JSON parsing	LLM outputs malformed JSON, markdown fences, partial truncations	Silent pipeline failures; blank screens; unrecoverable errors
Hallucinated content	Scenes reference entities, locations, characters absent from the premise	Story incoherence; ungrounded narrative output

Problems Solved in This Build

#	Problem	Solution
1	Stories always rejected — "Max retries exceeded"	4-mode diff-patch repair (cycle · schema · structural · style)
2	FAISS 75+ false advisory warnings per run	Threshold calibrated to 0.20; per-scene score table in UI
3	FAISS rules genre-agnostic	5 genre-specific rule files; per-genre vault cache
4	Stories lost on server restart	Atomic JSON file store; survives Docker restarts
5	LLM quota errors → immediate fallback	Per-provider exponential backoff (2s) before chain escalation
6	Static spinner — no pipeline visibility	SSE live progress stream; per-agent stage messages
7	Rejection banner said "0 issues detected"	Precise banner naming exact failure causes
8	IBM Guardian not connecting (region, key format)	URL → us-south; UUID key only; model → granite-guardian-3-8b
9	No hallucination detection	NEW HallucinationVerifierAgent — RAG confidence + grounding

---

3. System Architecture

```
BROWSER  http://localhost:5173
  React 18 + Vite | ReactFlow story tree | Recharts cost dashboard
  3D Blueprint | Emulator | useProgress SSE hook | Export
        |
        |  POST /api/generate     ← generate story (pre-flight budget check)
        |  GET  /api/progress/id  ← SSE live progress stream
        |  GET  /api/stories/id   ← retrieve stored story
        ▼
FASTAPI  :8000  (nginx reverse-proxy, 120s timeout)
        |
        ▼
```

4. Agent Pipeline — All 7 Agents

Agent 1 — Story Generator

File	`agents/story_generator.py`
Purpose	Generate branching narrative JSON from premise + genre + tone
LLM Chain	Groq llama-3.3-70b-versatile → Groq llama-3.1-8b-instant → Gemini gemini-2.5-flash
Output	12–25 scenes, DAG-structured, tone-labelled, JSON validated
Repair Modes	cycle_repair · schema_repair · structural_repair · style_repair (priority order)
Zero-token Sanitisers	`_sanitise_schema()` + `_fix_dangling_refs()` at parse time
Typical Duration	6–8 seconds (Groq primary, no retries)

Agent 2 — Hallucination Verifier NEW

File	`agents/hallucination_verifier.py`
Purpose	RAG-style confidence + grounding check — every scene must trace back to the premise
Model	all-MiniLM-L6-v2 (sentence-transformers, cached locally)
Layer 1	Retrieval confidence: cosine similarity scene vs premise fragments; flag < 0.20
Layer 2	Evidence grounding: best-matching premise fragment stored per scene
Layer 3	Verification: novel proper nouns absent from premise penalise score (−0.05/entity)
Layer 4	No blind rejection: flags fed to repair loop; pipeline never blocked
Typical Duration	~1–3s (model cached); ~44s first run (model download)

Agent 3 — Style Vault

File	`agents/style_vault_agent.py + core/style_vault.py`
Purpose	Genre-scoped FAISS semantic similarity + deterministic tone gate
Blocking	Tone label mismatches (e.g., "playful" in thriller) — triggers style_repair
Advisory	FAISS cosine scores shown as colour bars in UI — never block approval
Rule files	3 common files + 5 genre-specific files (thriller, fantasy, sci-fi, educational, marketing)
Typical Duration	5–10 seconds

Agent 4 — Sandbox Validator

File	`agents/sandbox_validator.py + sandbox/validator.py`
Purpose	Structural and schema validation using NetworkX graph theory
Check 1	Schema: required fields, valid tone values, duplicate IDs, choice structure
Check 2	Cycle detection: `networkx.simple_cycles()` — extracts (source, target) back-edges
Check 3	Structural: dangling next references, orphaned scenes, minimum 6 scenes
Typical Duration	400–600ms

Agent 5 — IBM Granite Guardian

File	`agents/granite_guardian.py`
Purpose	Content safety scan via IBM watsonx.ai
Model	ibm/granite-guardian-3-8b (Lite plan, us-south region)
Categories	harm · violence · hate · sexual_content · profanity
Threshold	0.8 (configurable via GUARDIAN_THRESHOLD)
Scan volume	25 scenes × 5 categories = 125 API calls per run
Typical Duration	~104–140s (Lite shared endpoint); <30s (paid dedicated)

Agent 6 — Compliance Agent

File	`agents/compliance_agent.py`
Purpose	SHA-256 fingerprinted audit report + disk persistence
Output	Fingerprint stored in state; story persisted as JSON to `data/stories/`
Typical Duration	4–8ms

Orchestrator

File	`agents/orchestrator.py`
Purpose	LangGraph StateGraph wiring + budget-aware retry router
Routing	approved → guardian → compliance · retry (if budget) → generate · fail → compliance
Max Retries	2 (configurable via MAX_RETRIES)

5. State Fields (ScenePilotState)

Field	Type	Set by	Read by
`story`	dict	StoryGenerator	HallucinationVerifier, StyleVault, Sandbox, Guardian, Compliance
`hallucination_check`	dict	HallucinationVerifier NEW	Compliance, API route
`validation`	ValidationResult	StyleVault + Sandbox	Orchestrator, API route
`approved`	bool	Sandbox	Orchestrator, Guardian, API route
`broken_nodes`	list[tuple]	Sandbox	StoryGenerator (cycle repair)
`last_story`	dict	Sandbox	StoryGenerator (merge_patch base)
`structural_issues`	list[str]	Sandbox	StoryGenerator (structural repair)
`token_spend`	int	StoryGenerator	Orchestrator budget gate, API route
`budget_halt`	bool	StoryGenerator / Orchestrator	API route
`guardian_check`	dict	GraniteGuardian	Compliance, API route
`agent_spans`	list[dict]	All agents	ComplianceAgent, API route (UI)

6. Three-Layer Token Budget Gate

Layer	Where	What it does
Pre-flight	`api/routes.py`	Estimates worst-case cost before any LLM call; rejects with exact TOKEN_BUDGET_LIMIT=N recommendation
Router gate	`agents/orchestrator.py`	Checks remaining balance vs mode-aware reserve before each retry (1,200 repair / 9,500 full-gen)
Generator guard	`agents/story_generator.py`	Defence-in-depth: refuses LLM call if balance < reserve; surfaces last_story as best-effort

```
full_gen_est  = 400 + (premise_chars / 4) × SCENE_MULTIPLIER
repair_reserve = 1,200 × MAX_RETRIES
worst_case     = full_gen_est + repair_reserve
recommended    = ceil(worst_case × 1.15 / 500) × 500
```

---

7. Four-Mode Diff-Patch Repair Engine

Priority	Mode	Trigger	Token cost
1	**cycle_repair**	broken_nodes non-empty (graph back-edges)	~600 tokens
2	**schema_repair**	schema_errors non-empty (missing fields)	~400 tokens
3	**structural_repair**	structural_issues non-empty (orphans, dangling refs)	~500 tokens
4	**style_repair**	style violations present, graph structurally sound	~800 tokens

Zero-token pre-repair (runs before LLM on every pass):

- `_sanitise_schema()` — fills missing id, tone, text, choices fields deterministically
- `_fix_dangling_refs()` — redirects broken `next` pointers to nearest valid forward scene

Result: 82% token reduction on retry passes (~600 tokens vs ~9,000 for full regeneration)

---

8. API Reference

Endpoint	Method	Description
`/api/generate`	POST	Run full pipeline. Body: `{premise, genre, tone}`
`/api/progress/{story_id}`	GET	SSE live progress stream — per-agent stage events
`/api/stories/{story_id}`	GET	Retrieve persisted story JSON
`/api/validate`	POST	Sandbox-only validation (no LLM call)
`/api/blueprint`	POST	Generate 3D spatial layout for game engine export
`/api/samples`	GET	Demo library — 5 pre-built story samples
`/metrics`	GET	Prometheus metrics exposition

---

9. Configuration Reference (.env)

Variable	Default	Description
GROQ_API_KEY	required	Groq API key (primary LLM)
GEMINI_API_KEY	required	Google Gemini API key (fallback LLM)
TOKEN_BUDGET_LIMIT	40,000	Total token ceiling per pipeline run
MAX_RETRIES	2	Maximum self-correction retries
SCENE_MULTIPLIER	40	Pre-flight token estimate multiplier
LLM_RETRY_DELAY	2.0	Seconds before retrying a quota-hit provider
GUARDIAN_ENABLED	false	Enable IBM Granite Guardian content gate
WATSONX_API_KEY	—	IBM Cloud API key (UUID only, no ApiKey- prefix)
WATSONX_PROJECT_ID	—	watsonx.ai project ID (GUID)
WATSONX_URL	https://us-south.ml.cloud.ibm.com	watsonx.ai region endpoint
GUARDIAN_THRESHOLD	0.8	Risk score cutoff (0.0 safe – 1.0 harmful)
GUARDIAN_MODEL	ibm/granite-guardian-3-8b	Guardian model ID on watsonx.ai
HALLUCINATION_ENABLED	true	NEW Enable hallucination verifier
HALLUCINATION_CONFIDENCE_THRESHOLD	0.20	NEW Similarity floor for scene grounding
STYLE_SIMILARITY_THRESHOLD	0.20	FAISS advisory flag floor
SANDBOX_USE_DOCKER	false	Run sandbox in Docker container
CORS_ORIGINS	localhost:5173,3000	Allowed CORS origins

10. Technical Stack

Category	Technology	Purpose
Orchestration	LangGraph 0.2	Typed StateGraph, conditional routing, retry loop
LLM Primary	Groq llama-3.3-70b-versatile	Story generation (primary)
LLM Secondary	Groq llama-3.1-8b-instant	Story generation (fallback, separate quota)
LLM Tertiary	Gemini gemini-2.5-flash	Story generation (tertiary fallback)
Safety	IBM Granite Guardian 3-8b	Content safety scan via watsonx.ai
Hallucination	sentence-transformers all-MiniLM-L6-v2	Scene-premise similarity scoring

FAISS	faiss-cpu 1.7.4	Style rule semantic search
Graph Validation	NetworkX	DAG cycle detection + structural checks
API	FastAPI + Uvicorn	REST API + SSE streaming
Frontend	React 18 + Vite + TypeScript	UI, story tree, dashboard, emulator
Visualisation	React Flow + Recharts	Story tree canvas + cost charts
Metrics	Prometheus + Grafana	Per-agent telemetry + dashboard
Infrastructure	Docker Compose + nginx	Container orchestration + reverse proxy

---

11. Verified Production Results

Latest Run — "Quantum Convergence" (Sci-Fi, 25 scenes)

Agent	Duration	Status	Notes
Story Generator	6,842ms	✓	2,891 tokens · 0 cycles
HallucinationVerifier NEW	43,598ms*	✓	*First run (model download); ~1–3s thereafter
Style Vault	5,417ms	✓	0 violations
Sandbox Validator	463ms	✓	0 cycles
IBM Granite Guardian	139,997ms	✓	0 violations · 25×5=125 API calls
Compliance	8ms	✓	fp: 55b7f1ab2744b894...

Metric	Value
Tokens used	2,891 (7% of 40,000 budget)
Retries	0
Validation gates passed	Cycles: 0 · Schema: 0 · Tone: 0 · Style: 0 · Structural: 0
Final status	• APPROVED

---

12. Project Structure

```
scenepilot-ai/
├── agents/
│   ├── state.py                    LangGraph state TypedDict
│   ├── story_generator.py          Generation + 4 repair modes + sanitisers
│   ├── hallucination_verifier.py   RAG confidence + grounding check  [NEW]
```

```
│   ├── style_vault_agent.py      Genre-scoped FAISS (blocking + advisory)
│   ├── sandbox_validator.py      NetworkX cycles + schema + structural
│   ├── granite_guardian.py       IBM Granite Guardian content safety
│   ├── compliance_agent.py       SHA-256 audit + disk persistence
│   └── orchestrator.py           LangGraph graph + budget router + SSE
├── api/
│   ├── main.py                   FastAPI app, CORS, metrics mount
│   ├── routes.py                 /generate /progress /validate /blueprint
│   └── samples_route.py          /samples demo library
├── core/
│   ├── story_store.py            Atomic JSON file store
│   ├── progress.py               Per-run SSE event queue
│   ├── llm_client.py             3-provider fallback chain + retry
│   ├── style_vault.py            FAISS index builder
│   ├── telemetry.py              Prometheus metrics
│   ├── blueprint.py              3D BFS spatial layout
│   └── utils.py                  merge_patch() for diff-patch repair
├── data/rules/                   Style rule files (common + 5 genres)
├── frontend/src/                 React 18 + Vite UI
├── prometheus/prometheus.yml
├── docker-compose.yml
├── Dockerfile
└── requirements.txt
```

---

13. Export Formats

Format	Description	Use case
JSON	Raw story graph — scenes + choices	Any game engine or further processing
Markdown	Human-readable with tone annotations	Writers, reviewers, documentation
Twine (.tw)	Twee 3 notation	Import into Twine 2 / compile with Tweego
Unity C#	MonoBehaviour blueprint	Unity game engine integration
Unreal Engine	Blueprint JSON	Unreal Engine integration
Godot	GDScript Resource	Godot game engine integration

Made with IBM Bob